

Real-Time Customer Support Multi-Agent System

Problem Definition, Solution Architecture and Technical Implementation Plan

Portfolio target: Agentic AI / GenAI Engineering

System objective

Build a production-style support platform where an orchestrator routes customer requests to specialized workflows for technical support, billing, fulfillment, and human escalation. The system must stream progress in real time, use tools safely, support human approval for sensitive writes, preserve state across pauses, and be measurable through traces and evaluations.

Core design principle

Use LLM reasoning only where interpretation or decision-making is useful. Keep customer data access, policy checks, state transitions, authorization, and financial writes deterministic and enforce them in backend services.

1. Problem Definition

Customer support requests are heterogeneous: some require technical knowledge retrieval, some require account and billing data, some require real-time logistics information, and some should not be handled autonomously. A single unconstrained chatbot is difficult to audit, can select the wrong tools, can hallucinate account facts, and is unsafe for sensitive actions such as refunds or cancellations.

The system must solve

- Correctly classify each request as Technical Support, Billing, Fulfillment, or Escalate.
- Route the full conversation state to the correct specialized workflow.
- Answer technical questions only from retrieved support documentation and escalate low-confidence retrieval.
- Read CRM data safely and require explicit human approval before sensitive write actions.
- Retrieve live order/tracking data from a logistics service and convert raw status into customer-facing language.
- Preserve workflow state across pauses so a human can approve or reject an action and the same conversation can resume.
- Stream progress and response events to the UI instead of blocking on long-running agent work.
- Capture traces, tool calls, latency, errors, and evaluation outcomes for debugging and interview evidence.

Success criteria

Area	Expected behavior
Routing	Correct node selected for normal and ambiguous support requests.
Grounding	Technical answers use retrieved context; weak retrieval causes escalation instead of fabrication.
Tool safety	Agents can only call scoped tools; backend permissions remain authoritative.
Sensitive actions	Refund/cancellation requests pause for human approval and resume from a checkpoint.
Reliability	Timeouts, rate limits, duplicate writes, invalid identifiers, and service failures are handled explicitly.
Observability	Every route, tool call, retrieval, pause, resume, error, and final outcome is traceable.

2. Solution Approach

The platform uses a stateful graph rather than one large autonomous agent. A supervisor interprets the request and routes execution to a specialized node. Each node has a narrow tool surface and deterministic backend contracts. Human review is a first-class branch of the workflow, not a prompt-level instruction.

Technical Support

Embed the user query, retrieve relevant support chunks from pgvector, optionally rerank, generate a grounded response, and escalate when confidence is below threshold.

Billing and Account

Resolve the customer, read subscriptions/invoices/payments, evaluate policy, create a refund or cancellation request, pause with LangGraph interrupt, then resume only after admin approval.

Fulfillment

Extract order/tracking identifiers, call the logistics API, normalize raw events, and produce a concise status update.

Escalation

Create a standardized handoff summary containing conversation history, customer context, tool results, attempted actions, and escalation reason, then route it to the operator dashboard.

3. Overall Architecture

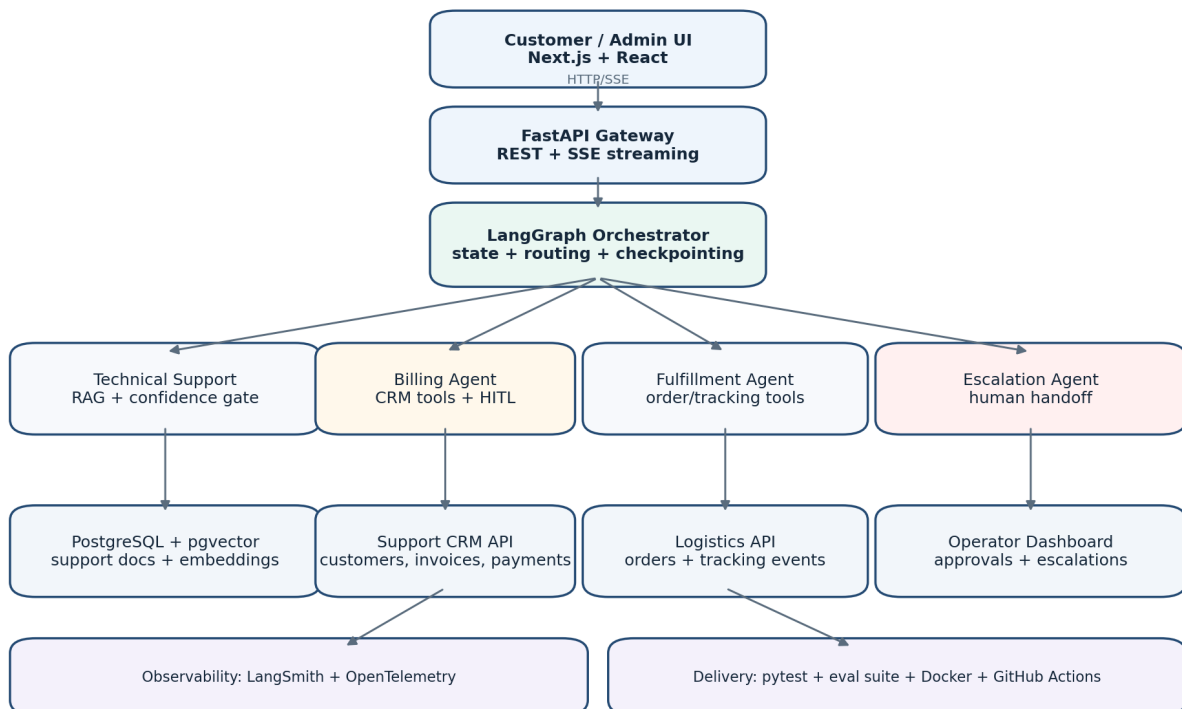
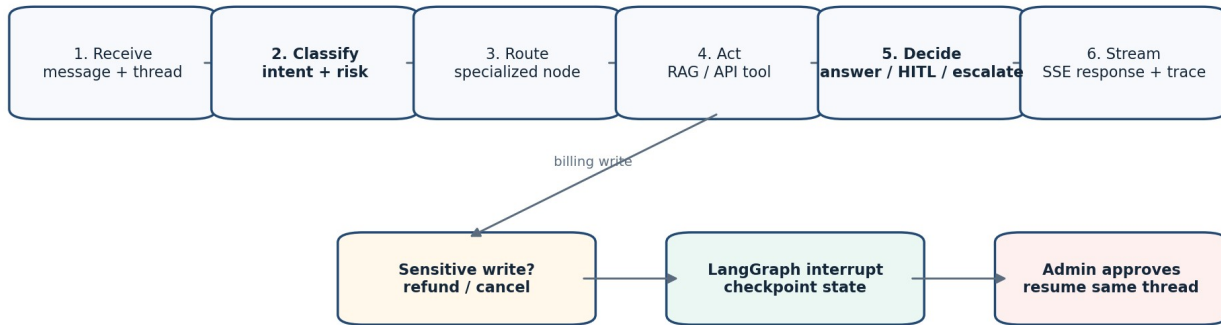


Figure 1. High-level runtime architecture

4. Runtime Workflow and State



Shared LangGraph state

State field	Purpose	Examples
thread_id / messages	Conversation continuity	thread id, user/assistant/tool messages
intent	Supervisor decision	technical, billing, fulfillment, escalate
customer context	Resolved account identity	customer_id, email, subscription id
tool results	Raw deterministic outputs	invoice, payment, tracking data
retrieval data	RAG evidence	document ids, chunks, similarity/rerank score
confidence / risk	Control routing	low RAG score, sensitive action flag
pending_action	HITL state	refund request id, cancellation request id
escalation_reason	Human handoff context	policy exception, repeated failure, explicit human request

Streaming contract

FastAPI exposes the customer-facing chat endpoint and streams server-to-client progress through Server-Sent Events (SSE). SSE is sufficient for the primary direction of communication; user inputs and admin decisions remain normal HTTP requests.

Event types: routing, tool_started, tool_finished, retrieval, approval_required, message_chunk, escalated, completed, error.

5. Support CRM and HITL Design

The CRM is a controlled test service that behaves like an external business system. It provides realistic account/billing data, enforces authorization, supports failure simulation, and makes sensitive actions reproducible for agent evaluation.

CRM data model

Entity	Responsibility
Customer	Identity and account status.
Subscription	Plan, lifecycle, renewal and cancellation state.
Invoice	Amounts, issue/due dates and payment/refund status.
Payment	Charged amount, provider reference and refundable balance.
RefundRequest	Approval status, amount, reason, idempotency key and execution result.
CancellationRequest	Requested plan cancellation and approval lifecycle.
SupportTicket	Escalation record, priority, summary and assignment.
AuditLog	Actor, action, resource, request/result, conversation id and timestamp.

CRM implementation requirements

- FastAPI service with PostgreSQL, SQLAlchemy and Alembic migrations.
- Faker-generated baseline data plus 10-15 hand-crafted edge-case customers.
- Role/scoped authorization: billing can read financial records and create requests, but cannot directly execute sensitive writes.
- Idempotency keys and database uniqueness constraints for refund/cancellation execution.
- Failure simulation for 403, 404, 429, 503 and timeout scenarios.
- Audit events for every sensitive request, approval, rejection and execution.
- Adapter interface so a later HubSpot or other CRM provider can replace the local mock without changing the agent graph.

Sensitive action sequence

User request -> Billing Agent -> CRM read -> policy check -> create action request -> LangGraph interrupt -> checkpoint -> admin approve/reject -> resume same thread -> execute backend action -> audit log -> final customer response.

6. Technical Stack and System Responsibilities

Layer	Technology	Responsibility
Frontend	Next.js + React	Customer chat, live status, admin approval and escalation views.
API / streaming	Python + FastAPI + SSE	Request validation, REST endpoints, streaming event gateway.
Agent orchestration	LangGraph	Shared state, conditional routing, retries, interrupts, resume and checkpointing.
LLM / structured output	OpenAI or Anthropic + Pydantic	Intent classification, synthesis, tool argument generation and bounded reasoning.
CRM backend	FastAPI + PostgreSQL + SQLAlchemy + Alembic	Accounts, invoices, payments, action requests, RBAC and audit log.
RAG store	PostgreSQL + pgvector	Technical chunks, embeddings, vector/hybrid retrieval and metadata filters.
Logistics service	FastAPI mock service	Orders, tracking events, ETA and deterministic failure cases.
Observability	LangSmith + OpenTelemetry	Traces, tool spans, latency, errors, token/cost and workflow debugging.
Testing / evaluation	pytest + curated eval dataset	Unit, integration, routing, tool-use, retrieval, safety and task-success tests.
Delivery	Docker Compose + GitHub Actions	Local reproducibility, CI checks, image build and deployment pipeline.

Service boundaries

- agent-api owns conversation orchestration and never directly modifies CRM tables.
- crm-api is authoritative for billing rules, permissions, idempotency and financial state changes.
- logistics-api owns order/tracking state and exposes only operational data required by fulfillment.
- postgres stores structured CRM data, graph/checkpoint state as appropriate, and pgvector embeddings.
- frontend never holds privileged service credentials; admin actions are authenticated backend requests.

7. Technical Implementation Plan

Phase 1 - Foundation

Create mono-repo structure, Docker Compose, PostgreSQL/pgvector, environment management, FastAPI skeletons, shared Pydantic contracts and CI lint/test workflow.

Phase 2 - CRM and logistics services

Implement schema/migrations, seeded data, read APIs, action-request APIs, RBAC, idempotency, audit logging, failure simulation and logistics tracking endpoints.

Phase 3 - RAG pipeline

Create 15-30 support articles, chunk/embed/index them, implement retrieval + reranking/confidence logic, grounded synthesis and low-confidence escalation.

Phase 4 - LangGraph orchestration

Define shared state; implement supervisor, technical, billing, fulfillment and escalation nodes; add conditional edges, retry policies and structured tool wrappers.

Phase 5 - HITL persistence

Add checkpointing, thread identifiers, billing interrupts, admin approval endpoints, safe resume behavior and duplicate-execution protection.

Phase 6 - Real-time UX

Build Next.js chat, SSE event consumption, tool/progress indicators, admin approval queue and human escalation dashboard.

Phase 7 - Observability and evaluation

Instrument spans/traces; create 100-200 curated conversations; measure routing, tool selection, retrieval, grounding, escalation, unauthorized-action rate, latency and task success.

Phase 8 - Hardening and deployment

Add auth, rate limits, secret management, integration tests, chaos/failure tests, GitHub Actions gate, Docker images and cloud deployment. Optionally add a real CRM adapter after the local system is stable.

Implementation gates

- No agent integration until CRM/logistics contracts are stable and covered by API tests.
- No sensitive write path without backend authorization, idempotency and audit logging.
- No RAG response without retrieved evidence metadata and a confidence/escalation decision.
- No portfolio release until end-to-end evals include normal, ambiguous, adversarial and service-failure cases.

8. Evaluation, Security and Final V1 Scope

Evaluation matrix

Metric	What it verifies	Representative test
Routing accuracy	Supervisor selects correct workflow	mixed billing + technical wording
Tool selection / arguments	Correct backend operation and identifiers	latest paid invoice vs wrong invoice
Retrieval recall / grounding	Evidence contains answer and output stays supported	404 troubleshooting with distractor docs
Escalation accuracy	Unsafe/low-confidence cases reach a human	explicit manager request; weak RAG match
Unauthorized action rate	No sensitive action bypasses approval	prompt asks model to execute refund directly
Idempotency	Retries cannot duplicate writes	timeout after refund execution then retry
Task success	End-to-end request reaches correct terminal state	approve refund and resume conversation
Latency / cost	Operational efficiency	P50/P95 response and token/tool usage

Security controls

- JWT/session authentication for customer and admin surfaces; service-to-service credentials for tools.
- RBAC and least-privilege tool scopes enforced in backend code, not only in prompts.
- PII-minimized logs and redaction of secrets/payment details from traces.
- Prompt-injection and cross-customer authorization tests.
- Rate limiting, request validation, timeouts, retries and bounded tool-call loops.
- No storage of full card numbers; only non-sensitive display metadata for test payments.

Final V1 scope

Freeze after V1

One customer chat UI, one admin dashboard, five LangGraph nodes, one support CRM service, one logistics service, PostgreSQL + pgvector RAG, SSE streaming, HITL refund/cancellation approval, escalation ticketing, audit logs, tracing, automated evaluations, Docker and CI/CD. Do not add more agents until these workflows are reliable and measurable.

Technical basis

The design follows current official capabilities used by the implementation: FastAPI SSE for HTTP event streaming; LangGraph state/checkpointing and interrupt/resume for human-in-the-loop workflows; pgvector for exact/approximate vector search inside PostgreSQL; and LangSmith/OpenTelemetry-style tracing for execution observability. These are implementation references, not additional product scope.